

SLDB: An End-To-End Heterogeneous System-on-Chip Benchmark Suite for LLM-Aided Design

Elisavet Lydia Alvanaki
Dept. of Computer Science
Columbia University
New York, New York, USA
ealvanaki@cs.columbia.edu

Kevin Lee
Dept. of Electrical Engineering
Columbia University
New York, New York, USA
kl3266@columbia.edu

Luca P. Carloni
Dept. of Computer Science
Columbia University
New York, New York, USA
luca@cs.columbia.edu

Abstract—Over the last few years, Large Language Models (LLMs) have emerged as a valuable tool for Electronic Design Automation (EDA). State-of-the-art research in LLM-aided design has demonstrated the ability of LLMs to generate syntactically correct RTL code, showcasing encouraging prospects for integrating AI into the hardware design process. A key enabler of these advancements is the availability of high-quality benchmarks to evaluate new approaches. However, existing datasets and benchmarks fall short of system-level design, as they focus primarily on component-level information and low-complexity designs. To address this gap, we introduce the System-Level Design Benchmark (SLDB), a dataset tailored for evaluating LLMs in system-level integration and configuration tasks. SLDB includes a curated benchmark suite of 10 baseline SoC designs, whose components can be combined into an exponential number of distinct tile-based SoCs through a synthetic library. The dataset provides full SoC configurations, accelerator integration code, communication parameters, and accelerator-aware system configurations, along with testing-application code, compatible with the ESP platform [1].

Index Terms—System-on-Chip, Electronic Design Automation, Benchmark, LLM-aided Design

I. INTRODUCTION

Modern hardware systems are becoming increasingly complex, requiring more efficient design methodologies that balance the trade-offs between performance, energy, and design effort. This complexity has increased with the rise of System-on-Chip (SoC) architectures, which integrate multiple, often heterogeneous components into a single chip to enhance performance across many application workloads. While heterogeneous SoCs yield significant gains in performance and efficiency, their design and validation require substantial engineering effort.

Optimized hardware components, designed at the register-transfer level (RTL), demand resource-intensive engineering efforts, characterized by long iteration cycles and extended time-to-market [2] [3]. These challenges are exacerbated for complex SoCs, underlining the need for more efficient design methodologies. In response to this scaling trend, agile design methodologies have emerged, incorporating modularity and design-space exploration techniques to accelerate the design process [4]–[6]. However, despite these advances, hardware

design remains constrained by manual, human-driven RTL development, limiting automation in the design flow.

Large Language Models (LLMs) have become a promising approach to addressing the growing complexity of hardware design. By leveraging prior designs, LLMs have shown notable results in the generation of RTL designs, making them an increasingly popular research domain in the context of agile hardware design. State-of-the-art works [7]–[10] have demonstrated the ability of LLMs to generate syntactically and *often* functionally correct code at the component level. As a result, two main challenges have emerged in *LLM-aided design*:

- 1) Establishing fair and comprehensive evaluation methodologies that assess LLM-generated hardware across diverse design aspects, including correctness, performance, and scalability.
- 2) Expanding LLM-aided design methodologies beyond individual components to complex system-level architectures.

To pave the way for addressing these challenges, we introduce the System-Level Design Benchmark (SLDB). SLDB is a curated dataset that enables the evaluation of LLM performance in system-level design of heterogeneous SoCs. In the short term, SLDB provides designers with a valuable resource to assess the performance of LLMs in various system-level design tasks, such as system integration and workload-aware end-to-end system configurations. Designers may use SLDB to evaluate their LLM-aided methodologies post-synthesis (*coarse-grain evaluation*) or to evaluate the ability of these methodologies to generate modular system-level abstractions (*fine-grain evaluation*), such as integration code and system-level design parameters. Designers may also use the SLDB synthetic library to validate more complex applications, dataflows, and component interactions. In the longer term, as LLM-aided design progresses, we envision SLDB serving as a solution to benchmark LLM-generated SoC designs against SoCs included in the dataset. SLDB features:

- Ten synthesized, heterogeneous SoC designs, integrating diverse workloads, accelerator-aware system configurations, and full-stack communication parameters.
- A synthetic library extension that seamlessly combines integrated accelerators to generate an exponential number of distinct SoC designs.

Our paper includes a comprehensive case study on LLM performance in modular SoC design using example design tasks from the open-source ESP platform [1]. This study demonstrates how using SLDB provides insights into LLM capabilities for hierarchical and scalable SoC generation.

II. RELATED WORK

Various works have introduced benchmarks and methodologies for evaluating the generation of RTL code. Koios is an advanced benchmark suite for Deep Learning EDA research [11]. Benchmarks and datasets specifically tailored for LLM-aided hardware design have been developed. In particular, Thakur et al. propose a fine-tuning methodology alongside a testbench-based evaluation framework to assess the functional and syntax correctness of LLM-generated Verilog code of homework-based design questions [12]. VerilogEval is an open-source benchmark using Verilog instructional resources as a data source [13]. Nazzal et al. propose a dataset for systolic array DNN accelerators [14] that is based on templated generation using Gemmini [15]. Zhang et al. propose criteria for data selection along with a Verilog dataset with corresponding natural language descriptions [16].

Moving towards larger-scale designs, RTLLM is an open-source benchmark for more complex RTL components, along with a prompt engineering methodology to enhance LLM-aided RTL generation [17]. RTL-Repo is a benchmark using repository-level hierarchy for multi-file RTL projects [18]. Met-Rex is a synthesizable benchmark to evaluate reasoning LLMs based on post-synthesis performance metrics [19].

To date, efforts in benchmark development have focused primarily on component-level tasks and lack the architectural complexity necessary to evaluate system-level design methodologies. While suggested *pass rates*¹ for LLM-generated hardware may appear promising, they fail to evaluate designs at the system level, where design parameters, interface specification, timing and communication play a critical role. Current evaluations are limited to self-contained hardware components, whereas realistic, scalable hardware systems functionally depend on the interplay between components, where specification adherence is crucial. *Without a structured system-level benchmark, it remains challenging to assess whether LLM-aided design can effectively scale beyond individual functional blocks to full heterogeneous SoCs.*

III. BACKGROUND

A. System-on-Chip Architectures

Modern SoC architectures feature a growing set of heterogeneous components, including specialized hardware accelerators that offer major performance and energy-efficiency gains [21]. These components impose diverse requirements in terms of area, communication, and delay, and yield distinct design-space tradeoffs. Designers have managed these trade-offs in different manners, aiming to achieve a balance

¹Pass rates (often referred to as *pass@k*) are defined as the percentage of tasks the LLM performs successfully in any of k independent runs (as defined in [20]). In LLM-aided design, a successful task is defined as a design that is functionally correct.

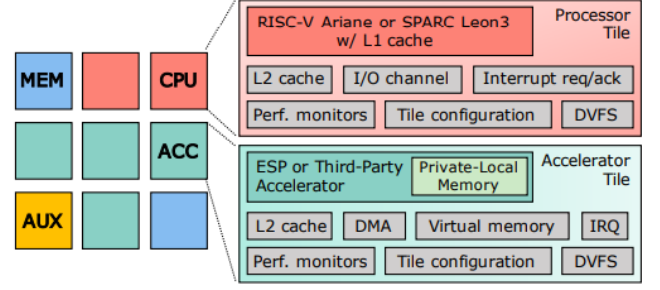


Fig. 1: The ESP tile-based architecture [1].

between performance and design effort. In the context of accelerator integration, these tradeoffs have resulted in two dominating paradigms: loosely coupled and tightly coupled accelerators [22].

Tightly coupled accelerators integrate directly with processor cores and memory hierarchies, significantly reducing latency. However, this tight integration limits flexibility, complicates scaling across workloads, and often requires custom instruction-set architecture (ISA) extensions and compiler modifications, thus increasing engineering effort.

In contrast, *loosely coupled accelerators (LCAs)* operate as independent processing units, typically communicating with the main processor and main memory via high-speed interconnects, such as networks-on-chip (NoC) and dedicated direct memory access (DMA) channels on the NoC. They often perform computation through four main distinct *Accelerator Processing Stages (APS)*: *Configure, Load, Compute and Store*, aimed at reducing communication overhead. During configuration, the accelerator receives the key computation parameters via memory-mapped configuration registers from the software application that has invoked it. Once the configuration is done, the load stage is initiated and the accelerator sends a DMA request to the memory and receives the requested data via the DMA channel on the NoC. Based on the acquired data, the accelerator performs computations and stores the result in memory via the DMA channel. Some LCAs feature *private local memories (PLMs)*, which are used to reduce NoC traffic. These accelerators are more scalable and modular, allowing SoCs to support a wide range of workloads.

While LCAs introduce higher communication overhead, they provide greater flexibility in heterogeneous computing, enabling optimized workload distribution and power-efficient execution. In developing the SLDB benchmark suite, we focused on modular SoC design based on LCAs and leveraged this modularity to enhance scalability and enable better LLM-design performance.

B. A brief overview of ESP

Due to their flexibility, LCAs have been widely adopted in SoC design. In tandem with these efforts, frameworks have emerged for agile system-level design, allowing designers to build correct-by-construction SoCs by combining many LCAs with processors and memories [4], [6]. To develop SLDB, we leveraged ESP, an established open-source platform for

Accelerator	Description	Code length	Dataset	Domain	Source Citation
AES Encryption	AES encryption algorithm	13736	RTL-Repo	Cryptography	[18]
AES Decryption	AES decryption algorithm	37658	RTL-Repo	Cryptography	[18]
SHA-256	SHA-256 encryption algorithm	13343	RTL-Repo	Cryptography	[18]
SOBEL	Edge detection filter	1951	RTL-Repo	Image Processing	[18]
FFT	64-point, 9-stage Fast Fourier Transform	200911	RTL-Repo	Image Processing	[18]
FCDNN	6-layer DNN with sigmoid activation	33163	RTL-Repo	Deep Learning	[18]
LSTM	Long Short Term Memory layer	69138	Koios	Deep Learning	[11]
SIMPLEDNN	7-layer DNN with relu activation	15420	RTL-Repo	Deep Learning	[18]
SPMV	MLP network for SPMV	111074	Koios	Deep Learning	[11]
CONVOLUTION	3-layer convolutional neural network	8937	RTL-Repo	Deep Learning	[18]

TABLE I: The characteristics of the ten accelerators selected to be part of the SLDB benchmark suite.

scalable SoC design and programming [1]. ESP integrates a modular, tile-based architecture with a flexible computer-aided design methodology that includes an RTL-driven design flow.

The ESP platform allows designers to specify the structure of a target SoC by configuring the number and type of SoC tiles across four main classes: CPU tiles, memory tiles, auxiliary tiles (with I/O peripherals such as UART and Ethernet), and accelerator tiles. Tile-to-tile communication is performed through a NoC, which supports DMA transfers.

Fig. 1 shows an example of the ESP tile-based architecture. ESP promotes design reuse of pre-designed intellectual property (IP) blocks, by seamlessly integrating them into different SoC configurations. For instance, a correctly designed accelerator IP can be integrated in many different ESP SoC design (e.g., SoCs with 2x2, 3x4 or 6x6 tile configurations) together with other accelerators or processing tiles, while remaining compatible with all system components. The integration of a heterogeneous component in the tile of a given SoC is performed through a *socket*, which is a parameterized, configurable interface. A socket interfaces the content of its tile with a multi-plane NoC and provides a variety of platform services [23]. The gray boxes in Fig. 1 illustrate such services—for example, a DMA engine and memory-mapped configuration registers in an accelerator tile socket, and an L2 cache and interrupt controller in a CPU tile socket.

While ESP facilitates the agile development of SoCs, most application-dependent design tasks must be performed manually. In particular, the SoC integration of a given accelerator requires the completion of two manual tasks starting from two provided templates: (1) the design of an RTL wrapper that interfaces the accelerator logic with the socket logic (including the DMA engine and the NoC interface) and (2) the implementation of a device driver to expose accelerator-specific parameters to software applications. The automatic completion of both these tasks is the target of the LLM-aided design techniques discussed in Section IV.

IV. THE SLDB BENCHMARK SUITE

A. Accelerator Data Acquisition

The SLDB benchmark suite includes complex accelerator designs from various domains, such as deep learning, cryptography and image processing. We selected a curated set of accelerators from the RTL-Repo and Koios benchmarks, to use as accelerators to be integrated into an SoC. We filtered

out programmable accelerators and accelerators that are not suitable for LCA integration. Our goal is to evaluate LLM performance on system-level integration as well as system configuration tasks. We opted to select some long-context, complex accelerators from the Koios benchmark, to assess the ability of the LLM to cater to designs with larger contexts. SLDB aims to evaluate LLM performance in realistic scenarios, with accelerators running complex and resource-intensive workloads. The selected designs illustrate diversity in terms of communication requirements, as well as workload properties. The structure of the dataset is illustrated in Table I.

We selected designs based on the following criteria:

Suitability for Loosely-Coupled Acceleration: Accelerators must be compatible with loosely-coupled integration, without direct dependencies on the processor’s ISA or tightly coupled memory hierarchies. This requirement ensures compatibility with ESP.

Workload Complexity: Accelerators reflect computationally intensive workloads, as used in modern SoCs. These workloads can effectively test integration and include diverse system configuration requirements.

Completeness, Quality, and Open-Source Availability: Selected accelerators must be complete, freely available, and adhere to good coding practices. Designs with incomplete source code, closed-source dependencies, or low-quality codebases were excluded to maintain the benchmark’s usability.

B. Integration into an ESP SoC

We structured the integration process into eight distinct stages, as shown in Fig. 2. Steps that require manual integration are denoted by circles with a fill, and automated steps are denoted by no fill. In Step ①, the accelerators are selected from the RTL-Repo and Koios benchmarks. In Step ②, the accelerator configuration is defined. In Step ③, the templates for the C testing application code (baremetal and Linux) are generated, along with a Verilog template for the integration of the RTL accelerator. The configurations are also written into the ESP files, to be used to generate the SoC in subsequent stages. In Step ④, the accelerator wrapper template is modified to integrate the accelerator. This includes the generation of RTL code to manage DMA handshakes, establishing mappings between the accelerator’s individual load, compute, and store operations and the packed data transfers performed via the DMA bus, as well as implementing the necessary timing and

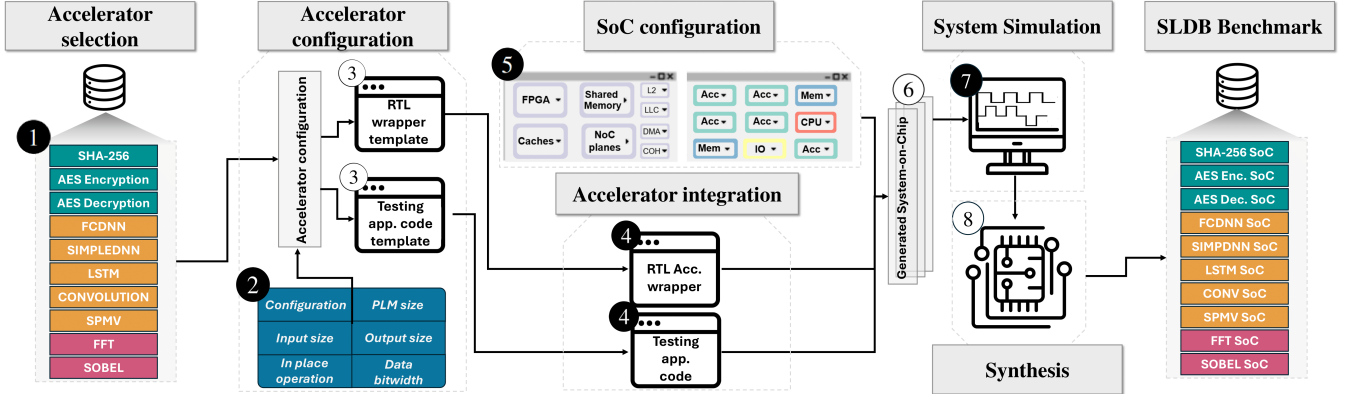


Fig. 2: SLDB design flow. Steps marked with a black-filled circle require manual configuration. Steps marked with a white-filled circle are generated by ESP.

SoC	Utilization (LUTs)	Static Power	Max Datapath Delay
AES Enc.	112,008	4.3 W	32.94 ns
AES Dec.	120,539	5.83 W	72.65 ns
SHA-256	99,699	2.87 W	10.01 ns
SOBEL	98,071	2.86 W	19.26 ns
FFT	109,627	3.39 W	3.8 ns
FCDNN	105,618	2.93 W	19.8 ns
LSTM	149,911	3.96 W	9.63 ns
SIMPLEDNN	98,518	2.88 W	9.59 ns
SPMV	103,940	2.94 W	32.94 ns
CONVOLUTION	120,311	2.9 W	19.37 ns

TABLE II: Performance and utilization of the ten SLDB SoCs.

internal synchronization logic to align accelerator operations with the Configure-Load-Compute-Store model. The testing application code is optionally modified to initialize the input and golden output values. In the context of this benchmark, we use randomized values to test the accelerators, since their functionality has already been verified. In Step ⑤, the heterogeneous SoC configuration is selected through the ESP GUI. The components are selected through drop-down options, with no code modifications required. In Step ⑥ the full SoC RTL is generated. In Step ⑦ the system is simulated and debugged to ensure functionality. Finally, in Step ⑧ the designs are synthesized for FPGA to retrieve performance, power and area (PPA) metrics.

C. The SLDB Benchmark Structure

The SLDB benchmark suite consists of ten heterogeneous SoCs. Each SoC has a *baseline* ESP 2x2 tile-based architecture, with four tiles: a CPU tile with the Ariane RISC-V core [24], a memory tile, an auxiliary tile and the accelerator tile, which contains one curated LCA from Table I. The SoC designs have been first evaluated in simulation, then synthesized and implemented with Vivado 2023.2 to retrieve PPA metrics. Table II reports post-implementation metrics.

We evaluated the correctness of the interfacing and SoC parameters, as well as the testing application code, but assumed correctness at the accelerator level, as the accelerators have been retrieved from pre-existing peer-reviewed works [11], [18]. Each design includes the full integration into an SoC using the design flow outlined in Fig. 2. The selected and avail-

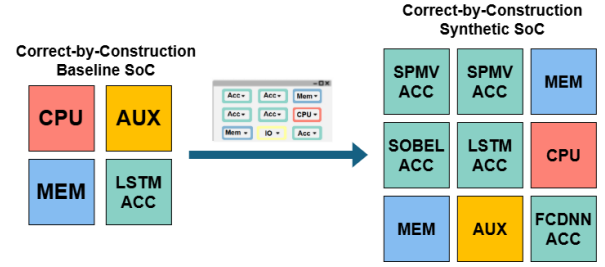


Fig. 3: SLDB synthetic library functionality.

able configurations for the SoCs are reported in Table III. The SoCs can run in bare-metal mode or with the Linux operating system, allowing designers to evaluate design methodologies that scale up to running software applications on top of the Linux operating system.

D. Scaling to different SoCs - The SLDB Synthetic Library

SLDB users may start from the baseline SoC configurations to create many different SoCs through the ESP GUI, as shown in Figure 3. Any combination of the baseline accelerators may be used with any amount or type of the ESP-compatible CPUs, memory, and auxiliary tiles. The available configurations are listed in Table III. Any configurations listed in the table can be selected along with the integrated accelerators without modifications to the existing code. The selection can be made by repeating Step ③ of the SLDB design flow and resynthesizing the design. Due to the correct-by-construction design capabilities of ESP, any SoC combination offers guaranteed correctness assuming a correctly integrated baseline accelerator and correct accelerator-specific configuration (provided in the benchmark).

V. CASE STUDY - SOC DESIGN WITH TASK DECOMPOSITION

We adopt a task decomposition [25] strategy and propose a *fine-grain* LLM evaluation approach structured around the ESP platform. This allows us to effectively assess the current capabilities of LLMs across diverse, modular hardware design tasks, while also enabling performance improvements through decomposition. Due to the complexity of end-to-end SoC design, we do not evaluate out-of-the-box LLMs in the full design task, as we consider current progress in the domain to

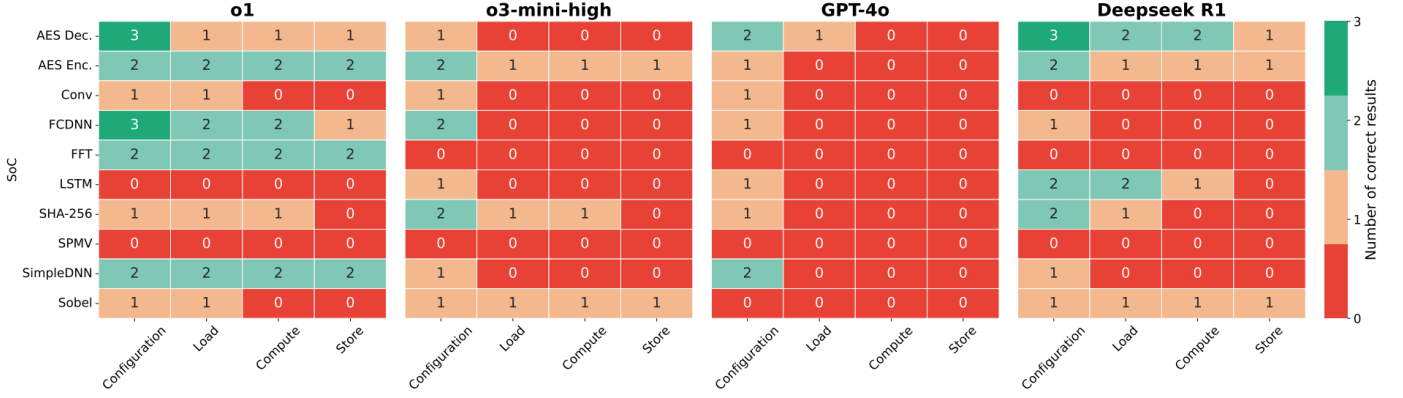


Fig. 4: Distribution of errors per Accelerator Processing Stage for 3 LLM generation runs.

Configuration	Selected / Range
FPGA Prototyping	xilinx-vc707 (profpga-xc7v2000t, xilinx-zcu102, xilinx-zcu106, profpga-xcvu19p, profpga-xcvu440, xilinx-vc707t, xilinx-vcu118, xilinx-vcu128)
CPU	Ariane (Ibex, SPARC)
Shared Local Memory (per tile)	256 KB (64-4096 KB, powers of 2)
Accelerator Data Allocation Strategy	Scatter/Gather (Big Physical Area)
Caches	No (LLC, L2, Accelerator L2 configurable)
SoC Dimensions (rows $\times$ columns)	2×2 (up to 8×8)
Coherence NoC Planes Bitwidth	64 (32-1024, powers of 2)
DMA NoC Planes Bitwidth	64 (32-1024, powers of 2)
Multicast	- (up to 16)

TABLE III: Selected system configuration parameters. Available options are illustrated in parentheses.

be inadequate for successful completion of the task without additional optimizations.

To explore the capabilities of current LLMs in SoC design aspects, we use SLDB to perform a case study of their functional and syntax correctness in a system-level design task. The LLM models are tasked with completing the integration of accelerators into ESP and evaluating required system parameters given the baseline accelerators. This case study evaluates:

- The ability of LLMs to abstract system-level parameters from RTL accelerators.
- The performance of LLMs in integration tasks with standardized protocols (such as DMA).
- The LLMs' ability to evaluate correct signal-port mappings and facilitate data transfers to and from the accelerator.

A. Evaluation Metrics

Functional correctness for each accelerator integration task is evaluated through the four sequential APSs defined in Section III. Each APS represents a critical step in ensuring accurate communication and integration of accelerators within

the ESP framework. The APSs evaluations are defined as follows:

- 1) *Configuration*: This stage evaluates whether the LLM-generated integration correctly assigns configuration parameters and initializes accelerator-specific registers required for correct operation.
- 2) *Load*: The correctness of data loading through DMA is evaluated by verifying if the accelerator correctly issues DMA requests, adheres to the appropriate handshake protocols, and receives the intended input data from memory.
- 3) *Compute*: This stage evaluates whether the integration code correctly maps accelerator input/output ports, accurately manages synchronization signals, and ensures proper computation within the accelerator logic.
- 4) *Store*: This stage assesses if the accelerator successfully returns computed results back to memory via DMA, correctly handling outgoing data and ensuring synchronization during transfers.

An integration is marked as fully functionally correct if all four APSs complete successfully. Correctness is assessed through simulation, and validated against integration by a human engineer. If a given stage fails, subsequent dependent stages are noted as failed due to dependency.

B. LLM Evaluation

We perform experiments to evaluate LLM performance using SLDB. The evaluated LLMs include a mixture of closed and open-sourced models, as well as reasoning and non-reasoning models. We evaluate OpenAI o1, GPT-4o, o3-mini-high and DeepSeek-R1-671B. The prompt used is identical across inferences and models. Due to space limitations, the full prompt will be included in the open-source release. Each LLM is tasked with generating DMA integration wrappers, extracting accelerator-specific configuration parameters, and generating driver code, for all ten SLDB accelerators. We conduct three independent inference runs per task, per accelerator, per LLM model and report pass rates (pass@1 and pass@3). We use the benchmark designs as our baseline.

Table IV compares the evaluated LLM models, presenting pass@1 and pass@3 metrics for syntactic and functional correctness. While syntactic correctness remains high, functional correctness varies across models and poses a significant challenge, with the highest pass rate at 50%. The pass@3

Model	pass@1		pass@3	
	Syntax	Functional	Syntax	Functional
o1	70%	40%	90%	50%
o3-mini-high	100%	10%	100%	20%
GPT-4o	60%	0%	70%	0%
DeepSeek-R1-671B	60%	0%	60%	30%

TABLE IV: Performance comparison of models for pass@1 and pass@3

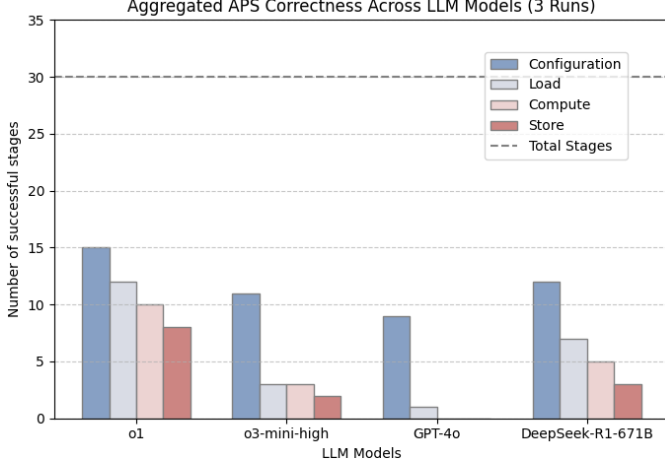


Fig. 5: Number of SoC integration errors per model. Experiments conducted using 3 independent inference runs.

metric significantly improves performance for some models, underlining the importance of multiple runs in fair evaluation of model performance.

APSS successfully completed over three inference runs are shown in Figures 4 and 5. Fig. 4 shows successful APSS for each SoC, per model. A value of 3 means that all generation runs were successful. We observe that o1 outperforms all models, yet still fails on more challenging accelerators, such as LSTM and SPMV. Fig. 5 shows the total number of successful APSS per model. It is evident that o1 and Deepseek R1 showcase minimal performance degradation across APSS, while other models show significant performance degradation in the load APS, where the DMA handshake begins. For Figures 4 and 5, the designs that complete the store APS successfully are also functional. *These results are cumulative over all design runs, whereas the pass@3 results consider non-overlapping successful design results across different LLM generation runs.*

Fig. 6 categorizes the types of errors encountered for each LLM across all runs. Prominent error types include incorrect handling of DMA signals, DMA state transitions, and specification adherence issues. Notably, advanced models like o1 and DeepSeek-R1 present fewer DMA signal errors, while most models struggle significantly with the complexity of state and control logic. Finally, specification adherence, such as active-low reset polarity, seems to become challenging for models as the length of the design code increases.

As shown in Fig. 5, while most models perform well in the configuration stage, their performance degrades in transition-defined APSS, such as the DMA handshake. The steep drop in success rates highlights the challenges LLMs face when

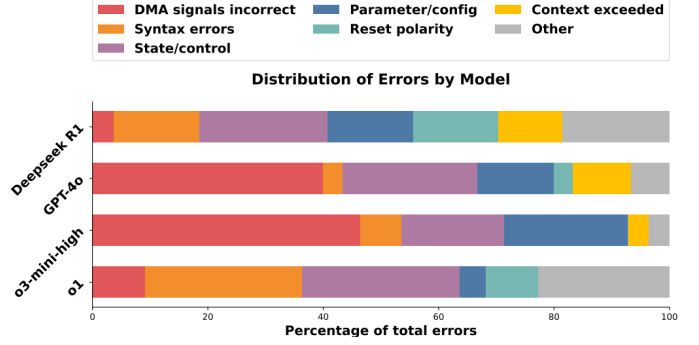


Fig. 6: Distribution of errors per model.

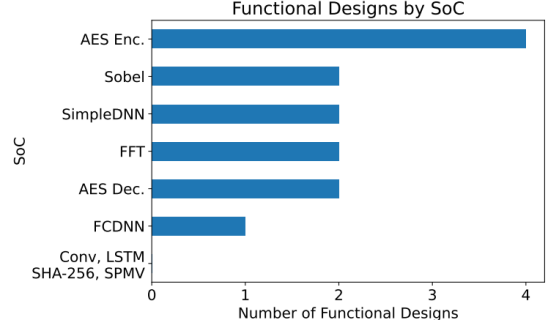


Fig. 7: Number of correct designs per SoC type

handling complex DMA handshakes and state transitions, suggesting areas for targeted improvements. Fig. 7 shows correctly generated accelerators, counted cumulatively for all models, across all runs. Results clearly indicate that certain accelerators, especially those with complex internal logic or advanced synchronization requirements (e.g., LSTM, SPMV, SHA-256), pose challenges to LLMs, underscoring areas for targeted improvement in future work.

VI. CONCLUSIONS AND FUTURE WORK

SLDB establishes a structured benchmark suite for evaluating LLM-aided SoC design at the system level. Unlike existing benchmarks, SLDB enables a comprehensive assessment of LLM performance in realistic and diverse system design tasks, including heterogeneous SoC configuration, accelerator integration, and system-level communication.

To demonstrate its utility, we conducted a case study on LLM-based fine-grain accelerator integration. Although LLMs achieved high syntax correctness, they struggled with system-level constraints, particularly DMA transactions, state transitions, and system-level parameter extraction. These challenges emphasize the need for specification-aware techniques such as streaming LLMs [26] to improve long-context understanding.

SLDB serves as a benchmarking tool for evaluating the performance of LLM-aided design methodologies and guiding future improvements in AI-driven hardware design. Future work in this direction includes expanding SLDB for broader system-level evaluations. SLDB is open-sourced² and will be maintained and improved as an open-source project to support continued research in LLM-aided system integration.

²<https://github.com/sld-columbia/SLDB>

VII. APPENDIX

A. In-Context Learning Evaluation

We perform an evaluation of the integration capabilities of LLMs through In-Context Learning (ICL), a technique that enables LLMs to leverage domain-specific context to improve performance on a given task [27]. We perform In-Context-Learning by providing the models with correct wrappers from the SLDB, and prompting them to complete the generation of template wrappers for the remaining designs. The following prompt is used for In-Context Learning: *"I will provide you with 3 reference accelerator wrappers, integrating 3 different accelerators into a system through DMA. Based on these wrappers, modify the input wrapper template that I will give you to integrate the input accelerator that I will give you. OUTPUT THE FINAL CODE AS IT WILL BE COMPILED AND TESTED. RESET FOR THE WRAPPER IS ACTIVE LOW."*

We use three accelerator wrappers (for the FCDNN, LSTM and SOBEL) as context, and evaluate the integration performance for the remaining seven accelerators: each accelerator is integrated in the SLDB baseline 2x2 tile SoC configuration described in Section IV-C. Table V reports the results of the ICL evaluation.

Model	Functional correctness	Syntax correctness
o1	0.00%	100%
o3-mini-high	14.20%	71%
GPT-4o	0%	71%
Deepseek-R1	0%	43%

TABLE V: Comparison of Model Performance with ICL

Figure 8 shows the distribution of the correctly generated APSs per model. The horizontal dashed line denotes the total number of SoCs, for which the LLM is tasked with completing the APSs. If a processing APS fails, then the subsequent APSs are classified as "failed" too, due to the strict functional dependencies among the four APSs. Figure 8 highlights each LLM model's capability to successfully integrate the four APSs using in-context learning – Configuration, Load, Compute, and Store. o1 exhibits the best progress, correctly completing six, four, and two APSs for configuration, load, and compute, respectively. GPT-4o displays a similar trend, but with lower overall performance. o3-mini-high and DeepSeek-R1-671B exhibit less completed APSs overall, each successfully completing only two APSs for the configuration, load, and compute APS. Notably, the o3-mini-high model is the only model that successfully integrated an accelerator from the configuration APS throughout the store APS. This analysis is a snapshot on the current level of performance exhibited by LLMs when applied to the task of integrating an LCA into an SoC using in-context learning. While o1 is the best performing LLM among the four evaluated models, it is clear that more progress remains to be made in this interesting avenue of research for the LLM-aided design community.

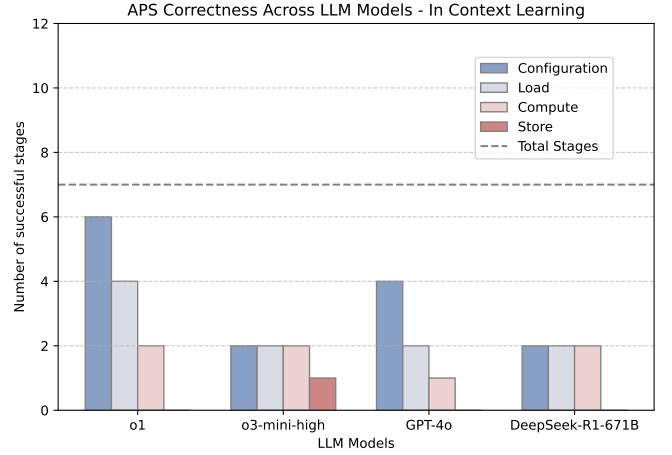


Fig. 8: Summary of correctly generated DMA stages, for 7 SoCs with In Context Learning.

REFERENCES

- [1] P. Mantovani, D. Giri, G. Di Guglielmo, L. Piccolboni, J. Zuckerman, E. G. Cota, M. Petracca, C. Pilato, and L. P. Carloni, "Agile SoC development with open ESP," in *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*, Nov. 2020.
- [2] B. Khailany, E. Krimer, R. Venkatesan, J. Clemons, J. S. Emer, M. Fojtik, A. Klinefelter, M. Pellauer, N. Pinckney, Y. S. Shao, S. Srinath, C. Torng, S. L. Xi, Y. Zhang, and B. Zimmer, "Invited: A modular digital VLSI flow for high-productivity SoC Design," in *Design Automation Conference (DAC)*, pp. 1–6, June 2018.
- [3] P. Rashinkar, P. Paterson, and L. Singh, *System-on-a-chip Verification: Methodology and Techniques*. Springer Science & Business Media, 2007.
- [4] A. Amid, D. Biancolin, A. Gonzalez, D. Grubb, S. Karandikar, H. Liew, A. Magyar, H. Mao, A. Ou, N. Pemberton, P. Rigge, C. Schmidt, J. Wright, J. Zhao, Y. S. Shao, K. Asanović, and B. Nikolić, "Chipyard: Integrated design, simulation, and implementation framework for custom SoCs," *IEEE Micro*, vol. 40, pp. 10–21, July/August 2020.
- [5] A. Rautakoura and T. Härmäläinen, "Does SoC hardware development become agile by saying so: A literature review and mapping study," *ACM Transactions on Embedded Computing Systems*, vol. 11, pp. 1–27, June 2023.
- [6] M. C. Dos Santos, T. Jia, M. Cochet, K. Swaminathan, J. Zuckerman, P. Mantovani, D. Giri, J. J. Zhang, E. J. Loscalzo, G. Tombesi, K. Tien, N. Chandramoorthy, J.-D. Wellman, D. Brooks, G.-Y. Wei, K. Shepard, L. P. Carloni, and P. Bose, "A scalable methodology for agile chip development with open-source hardware components," in *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*, ICCAD '22, Nov. 2022.
- [7] J. Blocklove, S. Thakur, B. Tan, H. Pearce, S. Garg, and R. Karri, "Automatically improving LLM-based Verilog generation using EDA tool feedback," 2025.
- [8] Y. Tsai, M. Liu, and H. Ren, "RTLFixer: Automatically fixing RTL syntax errors with Large Language Model," in *Proceedings of Design Automation Conference (DAC)*, DAC '24, 2024.
- [9] C.-T. Ho, H. Ren, and B. Khailany, "Verilogcoder: Autonomous Verilog coding agents with graph-based planning and Abstract Syntax Tree (AST)-based waveform tracing tool," *AAAI*, vol. abs/2408.08927, 2025.
- [10] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "RTLcoder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution," in *2024 IEEE International Workshop on LLM-Aided Design*, 2024.
- [11] A. Arora, A. Boutros, D. Rauch, A. Rajen, A. Borda, S. A. Damghani, S. Mehta, S. Kate, P. Patel, K. B. Kent, V. Betz, and L. K. John, "Koios: A deep learning benchmark suite for FPGA architecture and CAD research," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, pp. 355–362, 2021.

- [12] S. Thakur, B. Ahmad, Z. Fan, H. A. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking Large Language Models for automated Verilog RTL code generation," *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–6, 2022.
- [13] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: evaluating Large Language Models for Verilog code generation," in *International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [14] M. Nazzal, D. Vungarala, M. Morsali, C. Zhang, A. Ghosh, A. Khreishah, and S. Angizi, "A dataset for Large Language Model-driven AI accelerator generation," *ArXiv*, vol. abs/2404.10875, 2024.
- [15] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao, A. Ou, C. Schmidt, S. Steffl, J. Wright, I. Stoica, J. Ragan-Kelley, K. Asanovic, B. Nikolic, and Y. S. Shao, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proceedings of the 58th Annual Design Automation Conference (DAC)*, 2021.
- [16] Y. Zhang, Z. Yu, Y. Fu, C. Wan, and Y. C. Lin, "MG-Verilog: multi-grained dataset towards enhanced LLM-assisted Verilog generation," in *The First IEEE International Workshop on LLM-Aided Design (LAD'24)*, 2024.
- [17] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLML: An open-source benchmark for design RTL generation with Large Language Model," *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 722–727, 2023.
- [18] A. Allam and M. Shalan, "RTL-Repo: A benchmark for evaluating LLMs on large-scale RTL design projects," in *2024 IEEE LLM Aided Design Workshop (LAD)*, pp. 1–5, 2024.
- [19] M. Abdelatty, J. Ma, and S. Reda, "MetRex: A benchmark for Verilog code metric reasoning using LLMs," *ArXiv*, vol. abs/2411.03471, 2024.
- [20] S. Kulal, P. Pasupat, K. Chandra, M. Lee, O. Padon, A. Aiken, and P. Liang, *SPoC: search-based pseudocode to code*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [21] W. J. Dally, Y. Turakhia, and S. Han, "Domain-specific hardware accelerators," *Communication of the ACM*, vol. 63, p. 48–57, June 2020.
- [22] E. G. Cota, P. Mantovani, G. Di Guglielmo, and L. P. Carloni, "An analysis of accelerator coupling in heterogeneous architectures," in *2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2015.
- [23] L. P. Carloni, "Invited: The case for embedded scalable platforms," in *Proceedings of the 53th Annual Design Automation Conference (DAC)*, 2016.
- [24] F. Zaruba and L. Benini, "The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-ghz 64-bit risc-v core in 22-nm fdsoi technology," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, pp. 2629–2640, Nov. 2019.
- [25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, (Red Hook, NY, USA), Curran Associates Inc., 2022.
- [26] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," *arXiv*, 2023.
- [27] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems* (H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, eds.), vol. 33, pp. 1877–1901, Curran Associates, Inc., 2020.